DD_OSR-05 - Inventory Audit - Developer Implementation Guide

Version: v1.0 **Bundle:** 08_osr_equipment **Companion DD:** DD_OSR-05-Inventory_Audit-v1.0

1. Goal

Implement physical inventory audit and discrepancy resolution.

The service must let users:

- create an audit plan;
- assign count tasks;
- submit physical counts;
- compare against OSR-01 and OSR-INSTANCE expected stock;
- create discrepancies;
- approve and apply adjustments/write-offs through owning modules.

2. Runtime Flow

```
sequenceDiagram
  participant BO as Backoffice/Contractor App
  participant AUD as OSR-05
  participant OSR as OSR-01
  participant INST as OSR-INSTANCE-01
  participant APR as EM-CFG-04

  BO->>AUD: POST /api/inventory-audits
  AUD->>OSR: read balances at freeze time
  BO->>AUD: POST /generate-tasks
  BO->>AUD: POST /inventory-count-tasks/{id}/submit
  AUD->>INST: validate counted serials
  AUD->>AUD: create discrepancy
  BO->>AUD: propose resolution
  AUD->>APR: evaluate/start approval if needed
  APR-->>AUD: approval callback
  AUD->>OSR: POST /api/stock-movements
  AUD->>INST: POST lifecycle event if serialized write-off
```

3. Step 1 - Migrations

Create:

- `inventory_audit_plan`
- `inventory_count_task`
- `inventory_count_line`
- `inventory_discrepancy`
- `inventory_resolution`
- `inventory_write_off_item`

Add uniqueness/idempotency safeguards:

- one open count task per audit/location/sku/bin/counter combination;
- one active resolution per discrepancy unless previous resolution is rejected/cancelled;
- one applied resolution cannot be applied twice.

4. Step 2 - Create Audit Plan

Endpoint:

```
POST /api/inventory-audits
Idempotency-Key: audit-wik-central-20260731
Content-Type: application/json
```

Request:

```
{
  "operatorCode": "WIK",
  "auditType": "CYCLE_COUNT",
  "locationId": "WIK-CENTRAL-NRB",
  "scheduledStartAt": "2026-07-31T08:00:00Z",
  "scheduledEndAt": "2026-07-31T17:00:00Z",
  "scope": {
    "skuIds": ["WIK-ONT-HUAWEI-EG8145V5"],
    "binCodes": ["A-01", "A-02"]
  }
}
```

On OSR-05:

1. Validate request.
2. Check location through OSR-01 `GET /api/stock-locations/{location_id}`.
3. If contractor location, validate contractor through EM-02.
4. Create `inventory_audit_plan` in `DRAFT` or `SCHEDULED`.
5. Emit `InventoryAuditCreated`.

5. Step 3 - Generate Count Tasks

Endpoint:

```
POST /api/inventory-audits/iap-001/generate-tasks
Content-Type: application/json
```

Flow:

1. Load audit plan.
2. Query OSR-01 balances for scope:

   ```
   GET /api/stock-balances?operatorCode=WIK&locationId=WIK-CENTRAL-NRB
   ```

3. For serialized SKUs, query OSR-INSTANCE expected serials:

   ```
   GET /api/equipment-instances?operator=WIK&locationId=WIK-CENTRAL-NRB&state=IN_MAIN_WAREHOUSE
   ```

4. Store expected qty on generated task/line snapshot.
5. Assign counters.
6. Mark plan `IN_PROGRESS`.

The expected quantity must be frozen. If stock movements happen after freeze, they should be visible as post-freeze movements, not silently change expected qty.

6. Step 4 - Submit Count

Endpoint:

```
POST /api/inventory-count-tasks/ict-001/submit
Content-Type: application/json
```

Request:

```
{
  "lines": [
    {
      "skuId": "WIK-ONT-HUAWEI-EG8145V5",
      "countedQty": 97,
      "serialsCounted": ["48575443ABCD1234"],
      "counterComment": "Three units missing from bin."
```

```
    }
  ]
}
```

Flow:

1. Validate task belongs to actor or actor has supervisor permission.
2. Validate count task is `ASSIGNED` or `IN_PROGRESS`.
3. Save `inventory_count_line`.
4. Calculate `varianceQty = countedQty - expectedQty`.
5. For serialized SKUs, compare submitted serials to expected serial snapshot/OSR-INSTANCE lookup.
6. If variance or serial mismatch exists, run Drools severity policy.
7. Create `inventory_discrepancy`.
8. Emit `InventoryCountSubmitted` and `InventoryDiscrepancyOpened`.

7. Step 5 - Drools Severity

Fact:

```
public record InventoryDiscrepancyFact(
    String operatorCode,
    String auditType,
    String locationId,
    String contractorId,
    String skuId,
    String discrepancyType,
    int quantityDelta,
    BigDecimal estimatedValue
) {}
```

Sample rule:

```
package com.sophix.osr.inventory

rule "Critical shortage for serialized high value stock"
when
  $f : InventoryDiscrepancyFact(discrepancyType == "SHORTAGE", estimatedValue >= 10000.00)
  $d : InventoryDiscrepancyDecision()
then
  $d.setSeverity("CRITICAL");
  $d.setRequiresSupervisorInvestigation(true);
end
```

8. Step 6 - Propose Resolution

Endpoint:

```
POST /api/inventory-discrepancies/idisc-001/resolutions
Content-Type: application/json
```

Request:

```
{
  "resolutionAction": "WRITE_OFF",
  "comment": "Not found after recount and camera review.",
  "items": [
    {
      "skuId": "WIK-ONT-HUAWEI-EG8145V5",
      "quantity": 3,
      "reasonCode": "LOST"
    }
  ]
}
```

Flow:

1. Load discrepancy.
2. Validate action is allowed for discrepancy type.
3. Calculate estimated value.

4. Run `inventory-resolution-approval.drl`.
5. If approval not required, mark resolution `APPROVED`.
6. If approval required, call EM-CFG-04 evaluate/request.
7. Store approval refs and mark `PENDING_APPROVAL`.

EM-CFG-04 evaluate sample:

```
{
  "operatorCode": "WIK",
  "moduleCode": "OSR-05",
  "actionCode": "INVENTORY_WRITE_OFF_APPROVAL",
  "subjectType": "INVENTORY_RESOLUTION",
  "subjectId": "ires-001",
  "context": {
    "resolutionAction": "WRITE_OFF",
    "estimatedValue": 18000.00,
    "currencyCode": "KES",
    "locationId": "WIK-CENTRAL-NRB",
    "skuId": "WIK-ONT-HUAWEI-EG8145V5",
    "quantity": 3
  }
}
```

Camunda note: OSR-05 does not start Camunda directly. EM-CFG-04 starts Camunda only for `BPMN` approval policies.

9. Step 7 - Approval Callback

Endpoint:

```
POST /internal/inventory-resolutions/ires-001/approval-outcome
Content-Type: application/json

{
  "approvalRequestId": "apr-wof-001",
  "outcome": "APPROVED",
  "decisionComment": "Approved after inventory investigation."
}
```

Flow:

1. Load resolution.
2. Verify approval request id.
3. If already processed, return 200.
4. `APPROVED` -> resolution `APPROVED`.
5. `REJECTED` -> resolution `REJECTED`.
6. Emit event.

10. Step 8 - Apply Resolution

Endpoint:

```
POST /api/inventory-resolutions/ires-001/apply
Content-Type: application/json
```

For quantity adjustment/write-off:

1. Validate resolution `APPROVED`.
2. Check not already applied.
3. Call OSR-01 stock movement with deterministic idempotency key.
4. For serialized items, call OSR-INSTANCE lifecycle event for each instance.
5. Mark resolution `APPLIED`.
6. Mark discrepancy `APPLIED` or `CLOSED`.
7. Emit `InventoryResolutionApplied`.

OSR-01 call:

```
{
  "operatorCode": "WIK",
  "movementType": "ADJUSTMENT",
  "reasonCode": "INVENTORY_WRITE_OFF",
  "skuId": "WIK-ONT-HUAWEI-EG8145V5",
  "quantity": -3,
  "sourceLocationId": "WIK-CENTRAL-NRB",
  "originRefType": "INVENTORY_RESOLUTION",
  "originRefId": "ires-001",
  "actorUserId": "usr-inventory-supervisor"
}
```

OSR-INSTANCE call:

```
{
  "toState": "DECOMMISSIONED",
  "reasonCode": "INVENTORY_WRITE_OFF_LOST",
  "originRefKind": "INVENTORY_RESOLUTION",
  "originRefId": "ires-001",
  "actorUserId": "usr-inventory-supervisor",
  "notes": "Approved write-off after physical count discrepancy."
}
```

11. Step 9 - Close Audit

Audit can close only when:

- all count tasks are submitted or cancelled;
- all discrepancies are closed, rejected, no-action, or applied;
- all required approvals have final outcome;
- all OSR-01/OSR-INSTANCE correction calls succeeded or are explicitly cancelled.

Endpoint:

```
POST /api/inventory-audits/iap-001/close
```

12. Worker Jobs

Job	Purpose
`inventory-outbox-publish`	Publish audit/discrepancy events.
`inventory-resolution-apply-retry`	Retry failed OSR-01/OSR-INSTANCE correction calls.
`inventory-count-reminder`	Notify assigned counters before due date.
`inventory-discrepancy-escalation`	Escalate stale discrepancies.

These can run in the shared OSR worker deployment.

13. Tests

- Create audit plan for valid OSR-01 location.
- Generate count tasks freezes expected qty.
- Submit count with zero variance creates no discrepancy.
- Submit count shortage creates discrepancy with severity.
- Serialized serial mismatch creates discrepancy.
- Write-off proposal calls EM-CFG-04 when configured.
- Approval callback is idempotent.
- Apply resolution calls OSR-01 once with idempotency.
- Serialized write-off calls OSR-INSTANCE lifecycle event.

- Closing audit blocked while discrepancy open.